CENG-322

Deliverable 3

FINAL DOCUMENT LAYOUT

Please submit **a single PDF document per group**. This means that the document should be professionally organized and have a uniform style throughout. It should look as though it came from one team, not 4 separate students. Please note that instructors may choose to run your submission through TurnItIn or compare the submission with other students from other sections for academic integrity purposes.

A single **PDF document** with the following sections denoted using page numbers, headers/footers and a table of contents:

Must follow the sequence below:

Use proper formatting

1. Document Title : Deliverable 3
2. Team Name: LIFE (*Light, Indoor air, Freshness, and Energy*)
3. Project Name : LIFE Environmental Solutions
4. Student's names and IDs : 11
5. Table of Contents (this is with team contract)
6. Brief description of the project, couple sentences.: Our project will have us create a system using hardware and software to be able to have readings of air quality in indoor environments, along with a heat signature to allow us to detect multiple people in the room or any indoor space.
7. Table below with signatures
8. Members Info and Participation. Effort was put on the project (0 – 100%), students will be marked based on their effort, i.e. if the team gets 8, and one student effort was only 50%, that student mark will be adjusted to 8 * 50% = 4

Name	Student Id	Github Id	Signature	Effort
Mohamed Ali	n01440760	MohamedAli0760	M.A.A	100%
Jonathan Akabuike	N01510573	JonathanAkaBuike0573	J.A	100%
Kieran Sharma	n01548225	KieranSharma8225	K.N.S	100%
Farhan Habibzai	N01610299	FarhanHabibzai	F.H.	100%

9. In the pdf, write the requirement and comment on it for each item.
10. GitHub Repo link. All members must contribute to the repo.
11. Verify the link is working.
12. Login functionality, I will use the following credentials to test your app:
    Email: aaa@bbb.com Password: *Admin101!*
13. Document any other login credentials I should use i.e. Admin vs. regular user. I should not need to send you an email to login to your app, or create an account.
14. *Verify you had created an account in the DB with credentials above. Take a screenshot showing the account above in the DB.*

    Our Login Credentials for the App : admin1@gmail.com Password: 1234567

15. Screenshot showing the Authentication and users logged in using gmail or their email accounts, i.e. below: some used their credentials, some used gmail account to login into the app.

16. You are working on sprint 3.
17. Describe in detail, the work that has been completed by each team member in this sprint only.
    Mohamed: I'm working on the Daily Standup and the Monday. Also, im adding in any strings or any other functionality to app
    Jonathan: Im working on the Monday and the feedback page of the app
    Kieran: I'm working on the Feedback Page of the app
    Farhan: Im checking to see of the app is running correctly and that all of the necessary details are in the app

18. Each member must have a minimum of 10 commits, counted starting Oct. 19 (Two-week Sprint).

19. Marks deducted for members who are not meeting the minimum 10 commits (i.e. if only 8 commits, 20% percent deducted, …etc.).

20. Number of commits by each member for this sprint only.

Mohamed: 10 commits
Jonathan: 10 commits
Kieran: 10 and more commits
Farhan:

21. Sprint goals, list sprint goals.**: The sprint goals we have for sprint 3 is to have the Real-Time Air Quality Dashboard on the app, to have Monitoring of Room Occupancy, Dashboard Overview and Device Control, add Authentication Flow Completion to the sprint and to have the Development Creation**

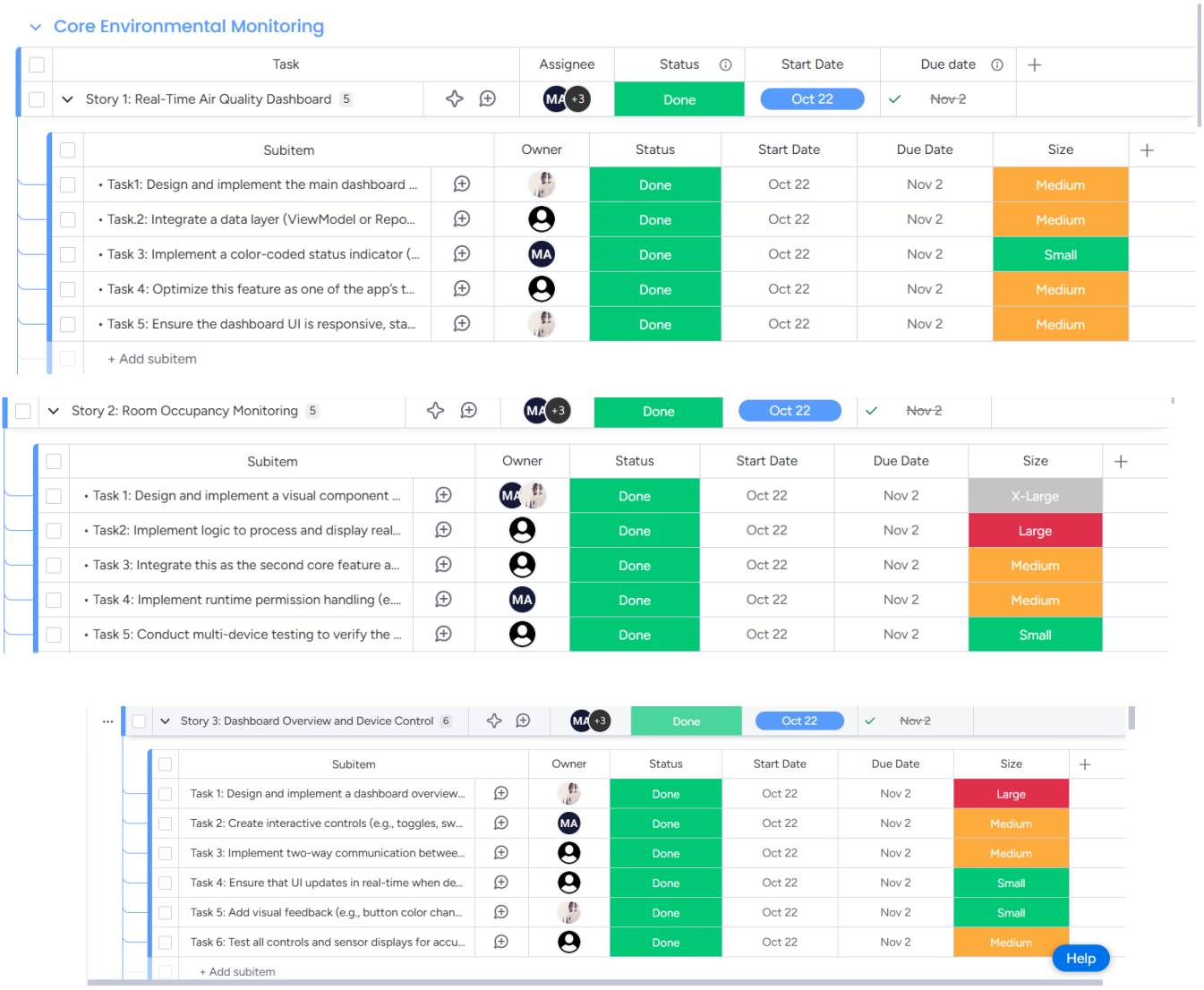
22. Update the Sprint dashboard, showing Sprint 3 with closed tasks and tasks you did not complete.

23. Dashboard, should clearly show task, owner, status, start date, end date, priority and size.

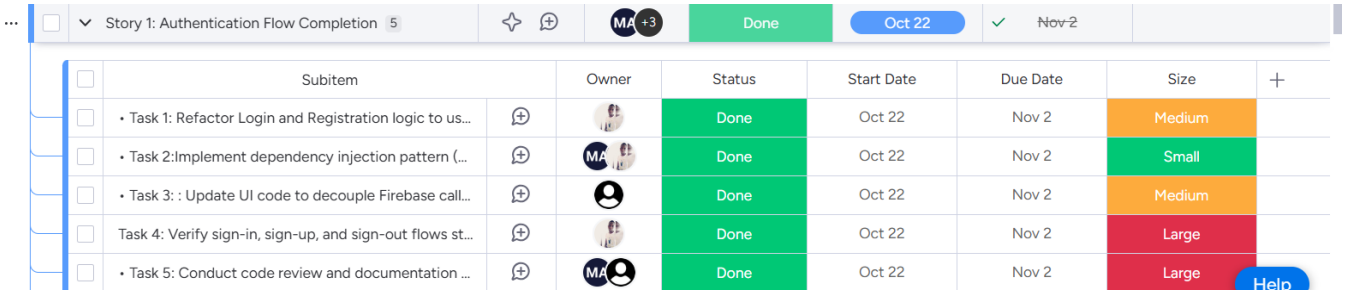
24. Must use a tool such as Trello, Monday …etc. Word, Excel, … etc. will not be accepted.

25. Must use Scrum or Kanban

Epic 1: Core Environmental Monitoring



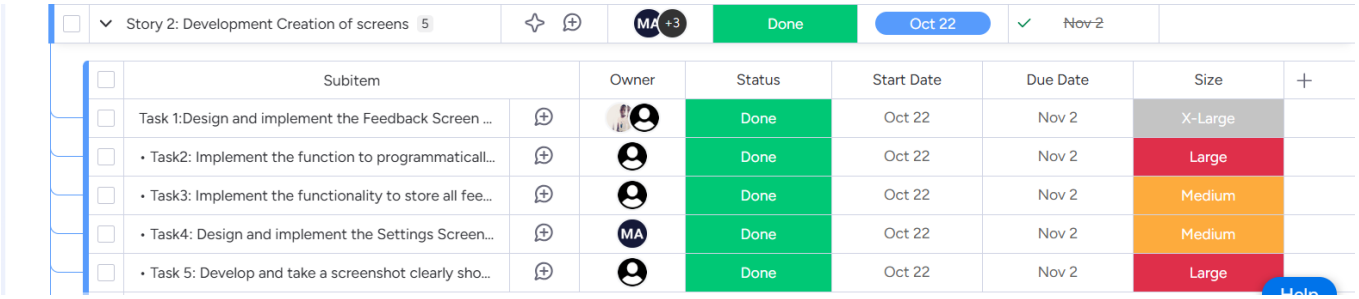
Epic 2: User and System Management



26. Take a screenshot showing clearly the stories and breakdown of tasks with all details for sprint 3 only.

27. Must have a minimum of 4 stories and 5 tasks per story.

28. Update gantt chart showing main milestones and work progress. You can use

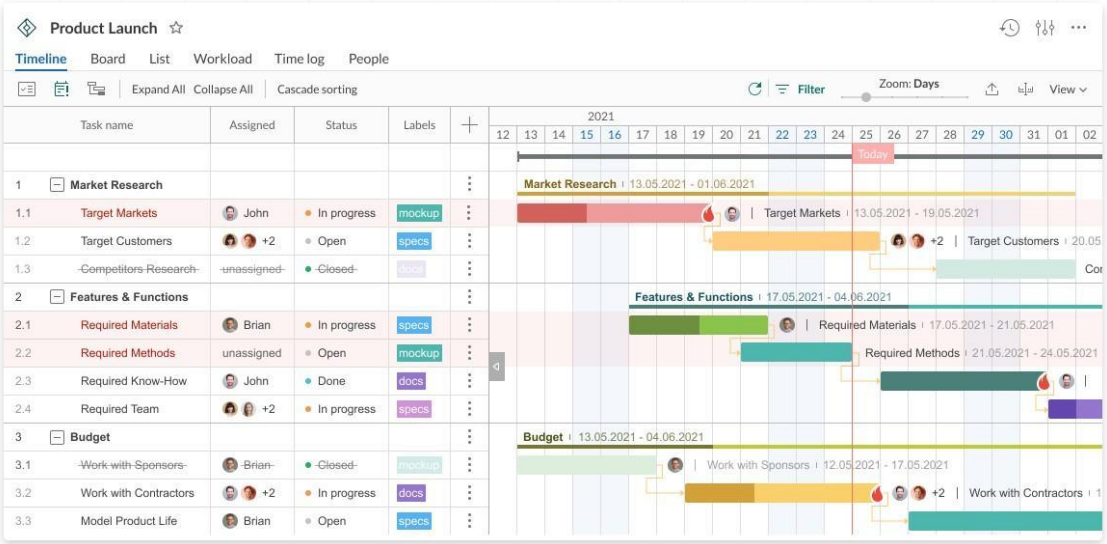


https://try.airtable.com/gantt2 or any other tool. Do the research **and select the** proper tool. Excel or word will not be accepted.

29. The gantt chart must show the work for a minimum of 10 components and the timelines. Focus on main components and not every small task.

30. Add a column to show the person assigned. An example of a Gantt chart:

31. Gantt chart should reflect the **entire project** from start to end. One of the main gaols for gantt chart to know the final delivery date for this term.



32. Take a screenshot of the gantt chart.

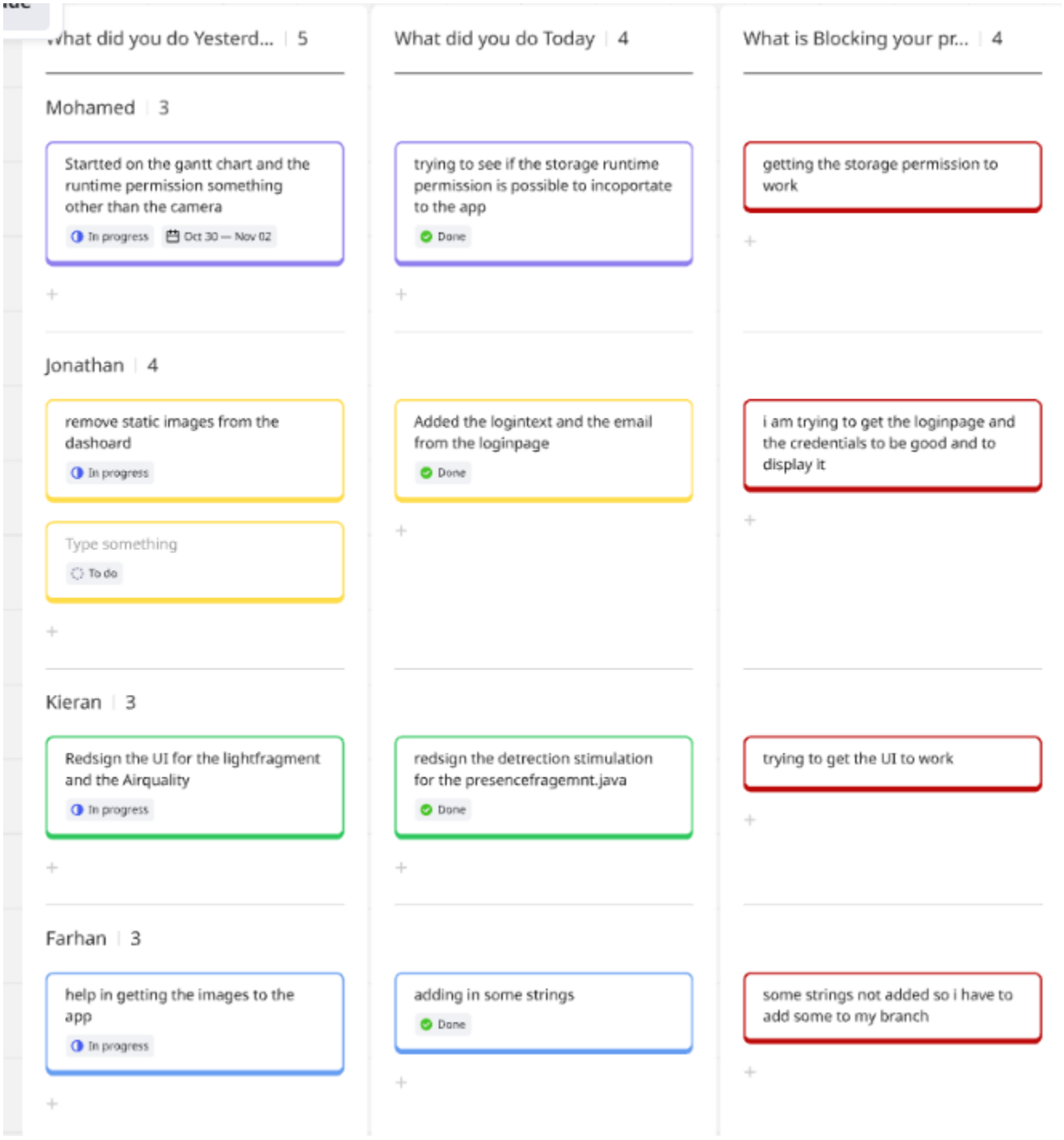
33. Updated records of the daily stand-ups outcome, use any tool you like, include a screenshot into the document. Table include date, outcome and who missed the daily standup.

34. Daily standup should answer three questions:
    1. What did you do yesterday?
    2. What will you do today?
    3. What (if anything) is blocking your progress?

35. Record a minimum of 3 meetings.

https://www.invisionapp.com

Meet on Oct 30, 2025



Meeting on Nov 1, 2025

Meeting on Nov 2, 2025

36. Document two different design principles used in the code. Copy the code you used, and add your explanations.

The first design principle is Open/Closed. This principle asslows us to have the core logic to calculate the sensor output if it is **closed**, while the parameters controlling that output are easily **extended**.

Code from SimulatedVEHL6030.JAVA

private Sample computeReading() { long now = System.currentTimeMillis(); // Base LSB (lux per count) for IT=100ms, gain=1 (arbitrary but stable) double baseLsb = 0.01; // 0.01 lux/count // Integration time scaling: uses 'it' object double itScale = 100.0 / it.ms; // Gain scaling: uses 'gain' object double gainScale = 1.0 / gain.factor; // **The core formula remains constant**

double lsb = baseLsb * itScale * gainScale; // Convert lux to raw counts, saturate at 16-bit int counts = (int)Math.round(trueLux / lsb);

The second one is Single Responsiliblity. This is going to have us responsible for modeling the behavior of an ambient light sensor and managing its internal state.

Code showing it on SmulatedVEHL6030.java
// simulation state private int minLux = 5, maxLux = 500; private double trueLux = 100.0; // slowly drifts within [minLux, maxLux] private double driftPerTick = 0.02; // small trend change per tick private Gain gain = Gain.GAIN_1; private IntegrationTime it = IntegrationTime.IT_100MS;

37. Document one design pattern used in the code. Copy the code you used, and add your explanations.
Design Pattern: Dependency Injection via Method Injection
Dependency Injection (DI) is a design pattern where an object's dependencies (i.e., the other objects it needs to work with) are "injected" into it from an outside source, rather than the object creating them itself.Method Injection is a specific type of DI where a dependency is supplied to a class through a method parameter. The class is only dependent on the dependency for the scope of that one method call.

- **This interface defines the common contract for all data-fetching strategies.**

```
public interface UserDataProvider {
   interface UserDataCallback {
      void onDataReceived(String userName);
      void onError(String errorMessage);
   }
   void fetchUserData(UserDataCallback callback);
}
```

- **This is a specific implementation of the strategy that fetches data from Firebase**.

```
public class FirebaseUserDataProvider implements UserDataProvider {
   @Override
   public void fetchUserData(UserDataCallback callback) {
      FirebaseUser currentUser = FirebaseAuth.getInstance().getCurrentUser();
      if (currentUser != null) {
         // In a real app, you might fetch from Firebase Realtime Database or
Firestore
         String userName = (currentUser.getDisplayName() != null &&
!currentUser.getDisplayName().isEmpty())
               ? currentUser.getDisplayName()
               : "User";
```

```
            callback.onDataReceived(userName);
        } else {
            callback.onError("User not authenticated.");
        }
    }
}
```

- The Method Signature (The Injection Point)The loadGreeting method doesn't create its own data source. Instead, it declares in its signature that it requires a UserDataProvider to be passed in.

```
public class DashBoardFragment extends Fragment {

    // ... (other fields)

    @Override
    public void onViewCreated(@NonNull View view, @Nullable Bundle
savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);
        // ... (other initializations)

        // The concrete strategy is created and passed in here (Method Injection)
        loadGreeting(new FirebaseUserDataProvider());
    }

    /**
     * This method depends on an abstraction (UserDataProvider), not a concrete
class.
     * It can work with any class that implements the UserDataProvider interface.
     */
    private void loadGreeting(UserDataProvider dataProvider) {
        // Use the injected provider to fetch data.
        dataProvider.fetchUserData(new UserDataProvider.UserDataCallback() {
            @Override
            public void onDataReceived(String userName) {
                // When data is received, update the UI.
                String firstName = userName.split(" ")[0];
                setGreeting(firstName);
            }

            @Override
            public void onError(String errorMessage) {
                // If there's an error, show a generic greeting.
                setGreeting("User");
```

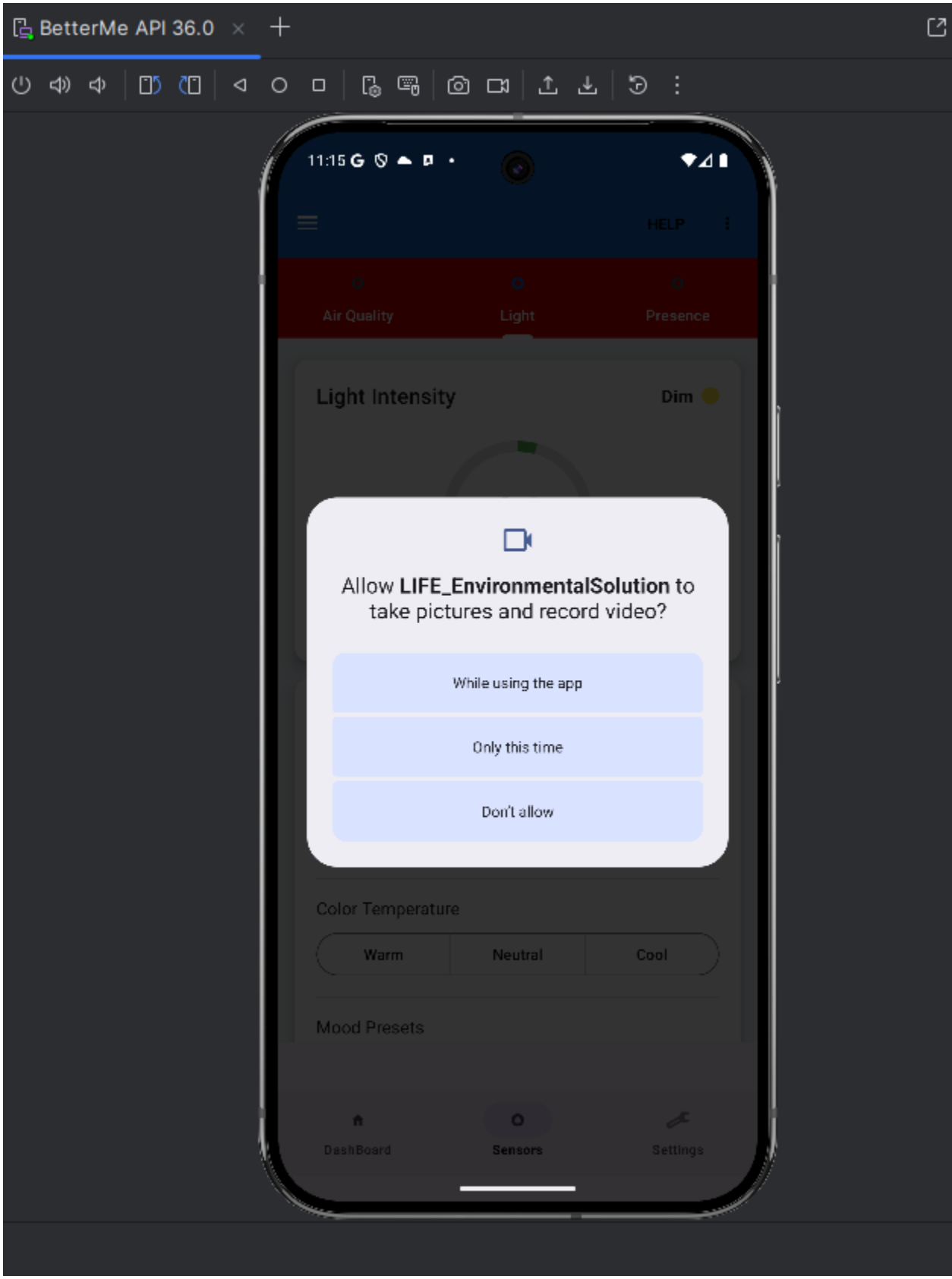
```
            if (getContext() != null) {
                Toast.makeText(getContext(), "Error: " + errorMessage,
Toast.LENGTH_SHORT).show();
            }
        }
    });
  }


}
```

Explanation.

Decoupling and Flexibility: The primary benefit here is decoupling. The DashBoardFragment (and specifically the loadGreeting method) is not tightly coupled to Firebase. Its only dependency is the UserDataProvider interface. This means you can easily change your data source in the future without modifying any of the logic inside loadGreeting. For instance, to switch to a different backend, you would just change one line: loadGreeting(new MyNewApiProvider());.

38. Document what runtime permission you have implemented, with a screenshot.:
the runtime permission here is the camera permission

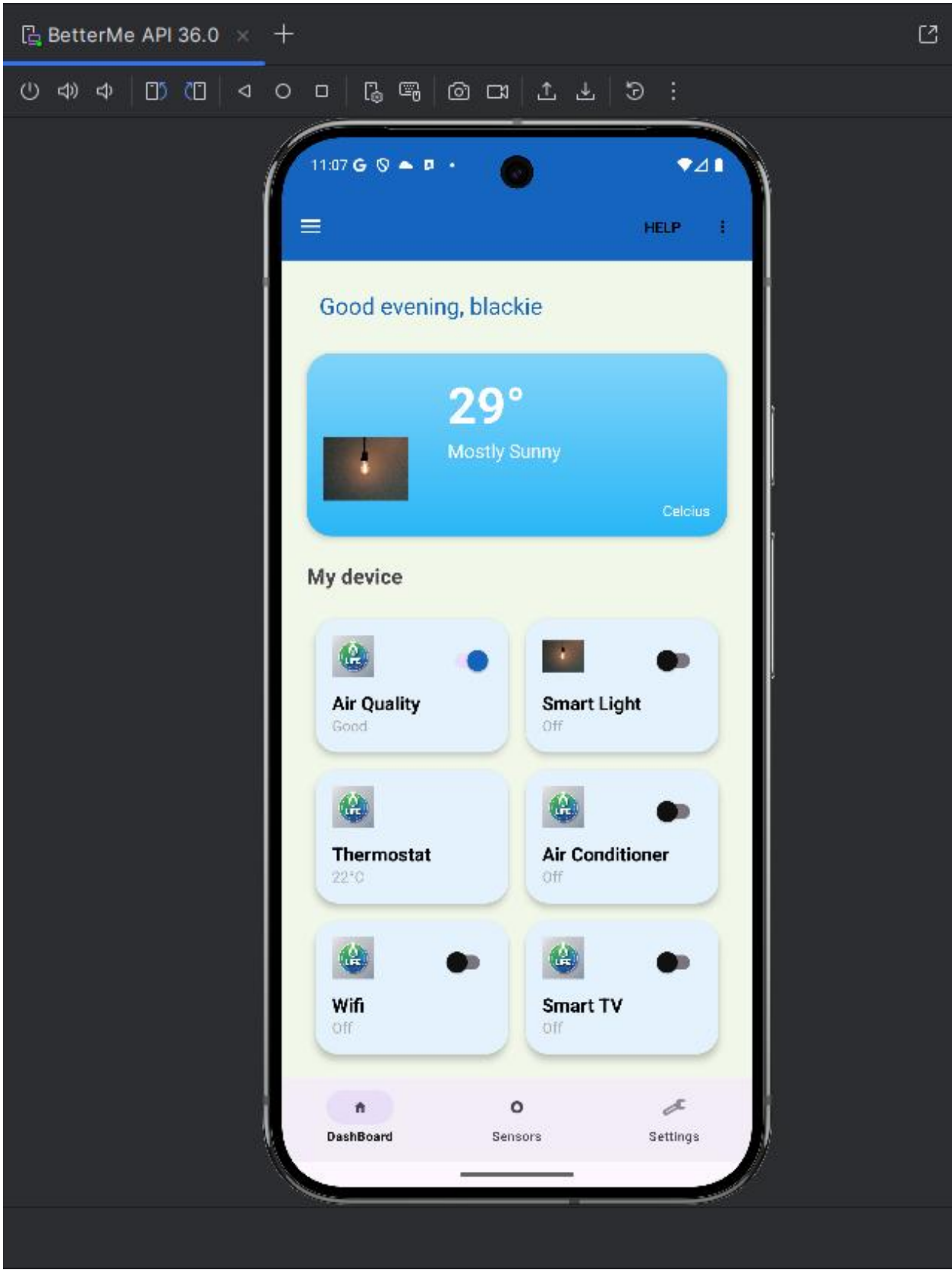


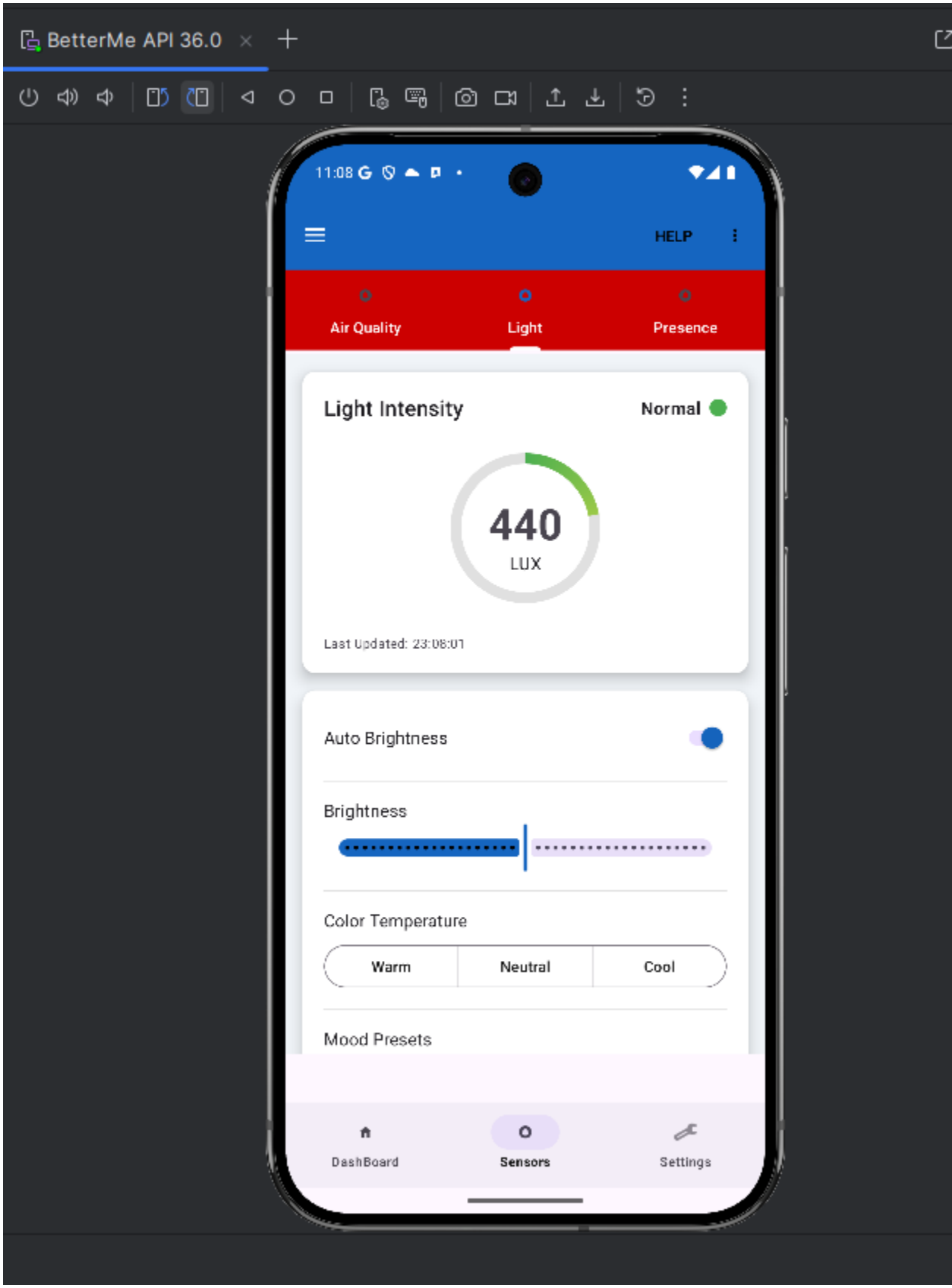
39. Document the two main functionality of your app that was implemented in this release. Take screenshots of the two features. Comment on the two screenshots.

Here is a brief documentation of the two main functionalities implemented in this sprint:

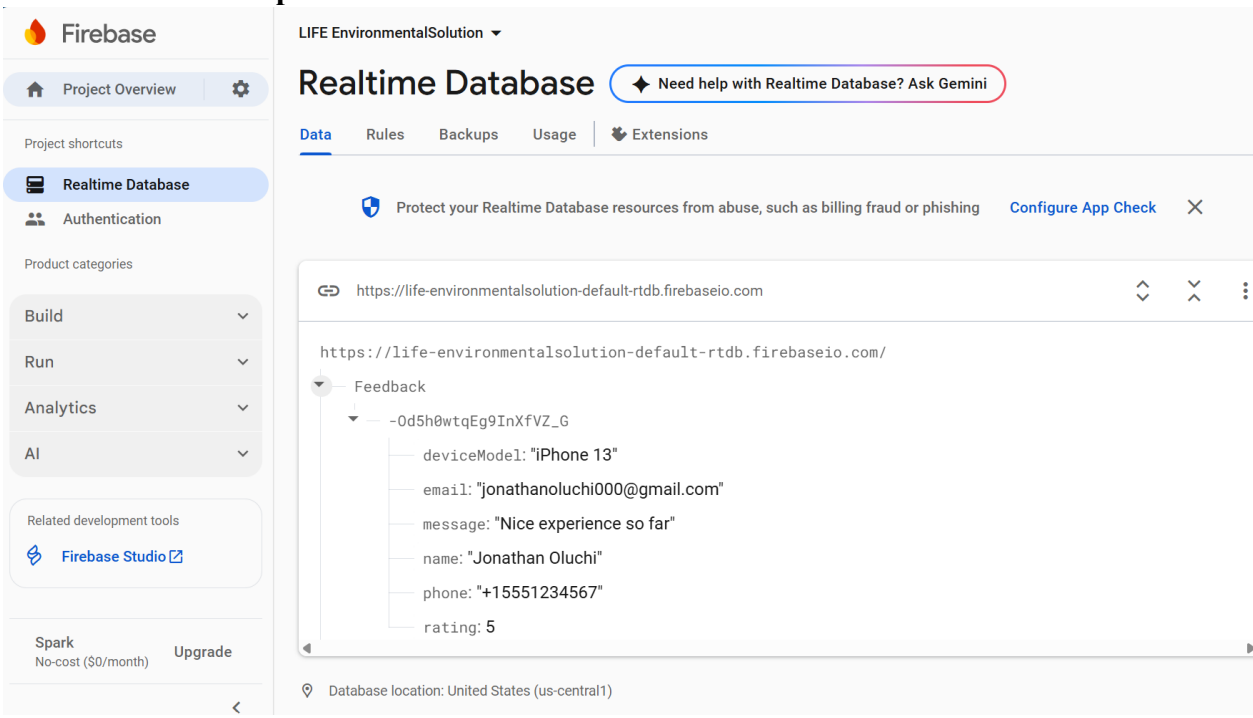
1.Advanced Control Panels for Smart Devices: Introduced detailed control modules for the Air Quality, Light, and Presence sensors. Users can now go beyond simple toggles to manage device-specific settings directly within the app. This includes adjusting brightness and color temperature for lights, setting CO2 alert thresholds for air quality, and configuring automated actions based on room occupancy.

2.Enhanced Dashboard & Startup Experience: the dashboard is now more dynamic, featuring a new weather card that integrates with a live weather API to display the current temperature and conditions.


BetterMe API 36.0
11:07
HELP
Good evening, blackie
29°
Mostly Sunny
Celcius
My device
Air Quality
Good
Smart Light
Off
Thermostat
22°C
Air Conditioner
Off
Wifi
Off
Smart TV
Off
DashBoard
Sensors
Settings


BetterMe API 36.0
11:08
HELP
Air Quality
Light
Presence
Light Intensity
Normal
440
LUX
Last Updated: 23:08:01
Auto Brightness
Brightness
Color Temperature
Warm
Neutral
Cool
Mood Presets
DashBoard
Sensors
Settings

40. **Provide screenshot how the data from the Customer Feedback Screen stored in the DB. See example.**



Screenshot showing the feedback from user saved to the database

41. Create a subfolder called deliverable3 under **docs** in the repo and add the pdf file.
42. Push the pdf into github.

If document not in pdf, or not the proper name (marks will be deducted!)
Please save the PDF document, i.e.
TeamName_ProjectName_GroupNumber_Deliverable3_.pdf One member to submit

the pdf file.

Deliverable 3:

Android Studio Project pushed into in github:

1- Use NavigaUon Drawer with the combinaUon with another layout.

2- Must Complete the UI design. All the UX must be completed.
3- Should have a minimum of 10 Java/Kotlin classes.
4- Each member must have a minimum of 10 commits. Counted starting Oct. 19.
5- App must have minimum of 7 screens (including the Splash, Login Registration, Feedback, Settings and Home Screen).

6- Splash, Login and Registration Screens must be activities and not Fragment. User access these screens prior to accessing the main application. All other screens must be fragments.

7- Once user login, they landed on the Home Screen.

8- Must dedicate a minimum of 2 screens for the main app functionality (excluding Settings, Feedback, Account, Profile, Help, …etc.).

9- Expect continuous commits for at least 8 days up to the delivery date.

10- Use proper name for the app name, don't use the word app.

11- Application name must be your project name.

12- Use proper name for the package name, avoid the words example, test,…etc. in the package name, otherwise your app will be rejected.

13- Use proper image name for the homescreen icons, replace the default android icon.

14- Minimum API 30 and target API 36/37.

15- Use two proper design principles.: Design Principle 1: Single Responsibility. Design Principle 2: Open/Closed Design Principle

16- Use one design pattern.: Design Pattern is Adapter Pattern

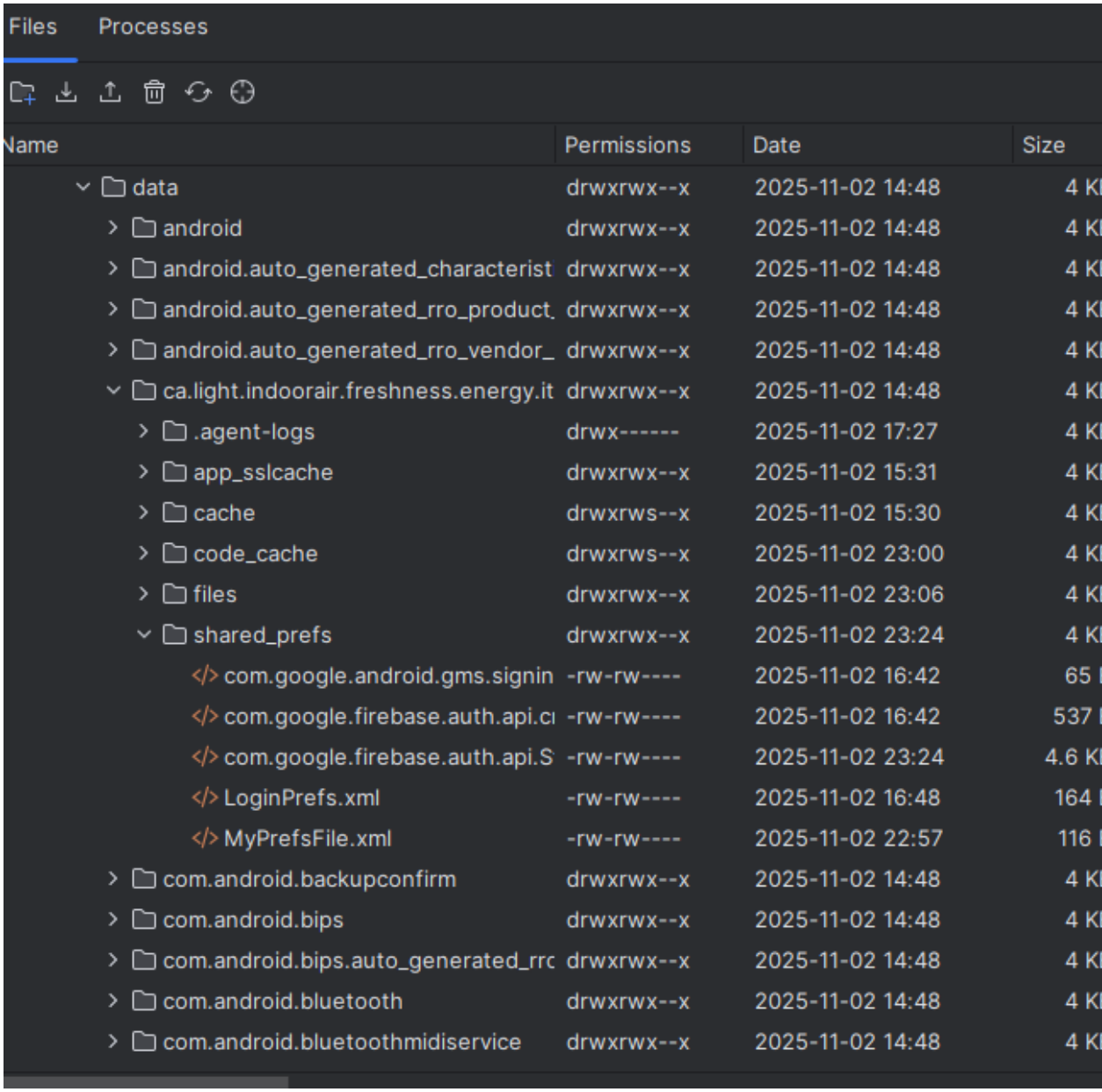
17- Complete the Top Bar (toolbar) menu implementation with all the functionality and UI. No further changes should be needed for the menu on the toolbar. Refer to Deliverable 2 for the requirements.

18- Complete the implementation of the runtime permission, with all the functionality.

19- Implement read and write from the cloud DB. Data read from DB displayed on screen. User update some data and update DB.

20- Should allow for configuration setting and remember user selections. Store data into SharedPref and read it back, i.e. if user selects Dark Theme, this option should be maintained when app restarts, device power off/on, …etc.

21- Take screenshot showing the data stored in the SharedPreference.



22- Complete the implementation of the settings screen.

23- Login logic, In addiUon to creaUng an account, allow the user to login using Gmail, without the need to create an account on your app

24- Login logic in your application, must have a checkbox “Remember Me”. Implement the functionality, so the user does not have to login every Ume access the app.

25- Login logic in your applicaU on, Complete the implementaUon of the registraUon and login screens.

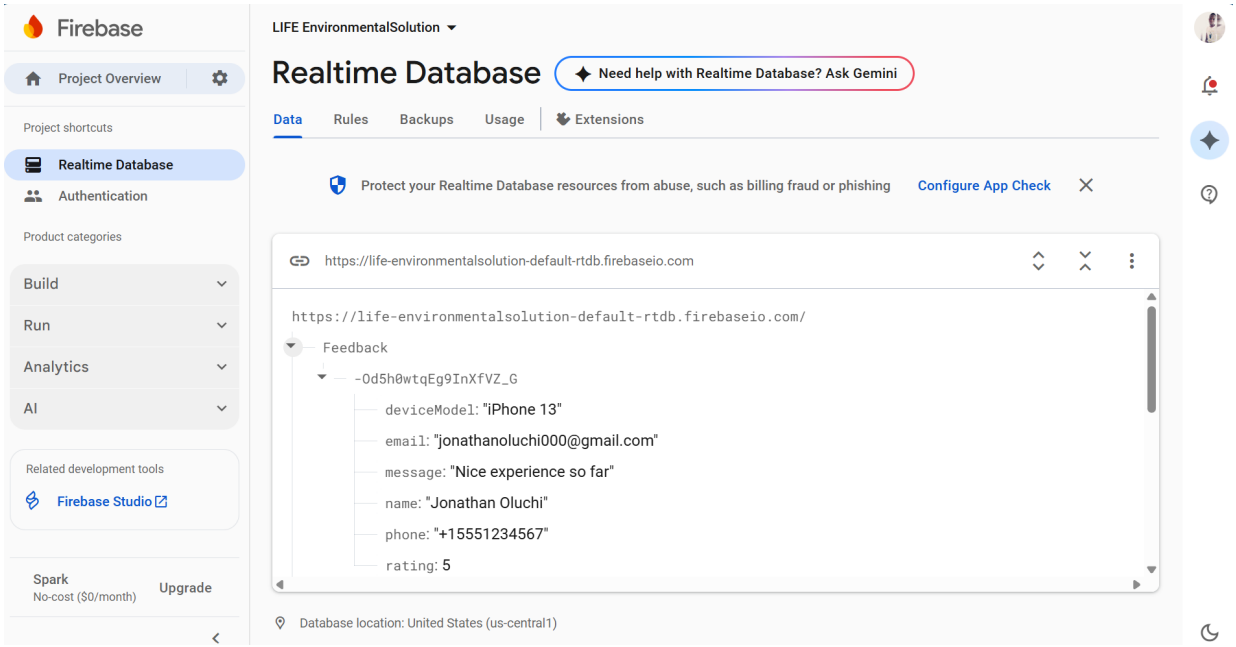
26- Registration Screen, verify registration screen has the following fields: Name, phone number, email, password and confirm password. See how to have a link on the login screen to the Signup screen.

27- Validate user input on registration screen, i.e. valid phone number, …etc. Must validate all user input, in addition restrict user input, i.e. phone number field should be numbers only, …etc

28- Remove username (userId) field. Email should be unique and used to identify users.

29- Feedback Customer relationship (Business Model Canvas): In addition to existing screens; design and implement a new "Customer Feedback Screen". https://www.pinterest.ca/masoodux/review-pageinspiration/

a. Use emoji or stars for the feedback.
b. Include, name, phone number, email and comment. Use proper formats for the entries, and the number of stars selected.
c. In addition to above read device model programmatically, and pass to the DB. Device model will not be shown on the feedback screen, however stored on the DB. https://www.tutorialspoint.com/how-to-check-android-phone-model-programmatically-using-kotlin
d. Store data into DB on cloud for assessment
e. Take a screenshot showing clearly the data transmitted from Feedback screen, and stored into DB.



30- Request Runtime permission completed. Request runtime when the functionality is requested, have proper design on the runtime.

31- The following screens, the design and functionality must be complete: splash screen, login screen, registration screen, feedback screen and settings screen.

32- Must complete the UI design and incorporate all the changes requested. No additional UI should be required for the subsequent submissions.

33- Must implement at least two main features of your application, they are related to the core functionality.

34- Plus deliverable 1 + 2.

35- App must not have compilation errors.

36- App must not crash.

37- App should be production grade app.

38- App will be tested on different simulators and devices.

39- Must not have an option in the Setting screen to switch Language. Language is set at the device level and not at the app level.

---------- **Deliverable 2:**

Refer to deliverable 2.